# Task 1 Tournament Ranking

105.173 Bayesian Statistics

Philipp Schott

2026-10-29

## Contents

**Data Description** **2**

**Bayesian Rating Model** **3**

Binary Model . . . . . 3

later (inkluding draws; idk, just experimental for now?) . . . . . 3

**Results** **4**

Engine Rankings . . . . . 4

Pairwise Superiority . . . . . 6

Tier List . . . . . 7

---

## Data Description

The first step is to load and inspect the provided dataset to understand its structure and verify its completeness before proceeding with the Bayesian rating model.

```
# Load dataset
games <- read.csv("../data/games.csv")

# Basic characteristics
str(games)
```

```
## 'data.frame':    862 obs. of  4 variables:
##  $ white      : chr  "A" "SM" "SM" "SM" ...
##  $ black      : chr  "SM" "A" "A" "A" ...
##  $ result     : chr  "1-0" "1/2-1/2" "1/2-1/2" "1-0" ...
##  $ timecontrol: chr  "60+1" "180+1" "60+1" "60+2" ...
```

```
cat("Missing values:" , sum(is.na(games)))
```

```
## Missing values: 0
```

The dataset contains 862 chess games played between different chess engines. Each observation represents one game and includes four variables: white, black, result, and timecontrol.

The variables white and black identify the participating engines and their respective colors. The variable result records the outcome of the game from White's perspective, while timecontrol specifies the time-control setting.

All 862 observations are complete, with no missing values across the four variables. Therefore, no missing-value imputation is required.

```
# Extract basic characteristcs
engines <- sort(unique(c(games$white, games$black)))

length(engines)
```

```
## [1] 6
```

```
engines
```

```
## [1] "A"  "B"  "C"  "D"  "E"  "SM"
```

```
table(games$result)
```

```
##
##     0-1     1-0 1/2-1/2
##     345     437      80
```

# Bayesian Rating Model

## Binary Model

### Data Preparation for Stan

```
# Engine labels
engines <- sort(unique(c(games$white, games$black)))

# First prototype: only decisive games
decisive <- subset(games, result %in% c("1-0", "0-1"))

# Stan requires numeric engine indices
stan_data <- list(
  N = nrow(decisive),
  K = length(engines),
  white = match(decisive$white, engines),
  black = match(decisive$black, engines),
  white_win = as.integer(decisive$result == "1-0")
)

str(stan_data)
```

```
## List of 5
##  $ N        : int 782
##  $ K        : int 6
##  $ white    : int [1:782] 1 6 6 1 1 6 1 1 1 1 ...
##  $ black    : int [1:782] 6 1 1 6 6 1 6 6 6 6 ...
##  $ white_win: int [1:782] 1 1 1 1 0 0 1 1 0 0 ...
```

### Posterior Sampling

```
# Find Stan file
stan_file <-"helpers/rating_binary.stan"

# Compile Stan model 1
mod_1 <- cmdstan_model(stan_file)

# Sample from posterior
fit_binary <- mod_1$sample(
  data = stan_data,
  seed = 42
)
```

## later (inkluding draws; idk, just experimental for now?)

### Data Preparation for Stan

```r
engines <- sort(unique(c(games$white, games$black)))

# Encode outcomes:
# 1 = White win, 2 = Draw, 3 = Black win
outcome_levels <- c("1-0", "1/2-1/2", "0-1")

stan_data <- list(
  N = nrow(games),
  K = length(engines),
  white = match(games$white, engines),
  black = match(games$black, engines),
  outcome = match(games$result, outcome_levels)
)

table(stan_data$outcome)
```

```
##
##   1   2   3
## 437  80 345
```

### Posterior Sampling

```r
stan_file <- "helpers/rating_three_outcomes.stan"

# Compile Stan model 2
mod_2 <- cmdstan_model(stan_file)

fit_full <- mod_2$sample(
  data = stan_data,
  seed = 42
)
```

# Results

## Engine Rankings

```r
# Create a ranking table from posterior rating samples
ranking_table <- function(fit, engines) {

  # Extract posterior samples for all engine ratings
  draws <- as.matrix(
    fit$draws(
      variables = "rating",
      format = "draws_matrix"
    )
  )

  # Summarize each engine's posterior rating distribution
```

```
  result <- data.frame(
    Engine = engines,
    Mean = colMeans(draws),
    SD = apply(draws, 2, sd),
    Lower = apply(draws, 2, quantile, probs = 0.05),
    Upper = apply(draws, 2, quantile, probs = 0.95)
  )

  # Rank engines by expected rating (highest first)
  result <- result[order(-result$Mean), ]
  rownames(result) <- NULL

  return(result)
}

# Generate separate rankings for both models
ranking_A <- ranking_table(fit_binary, engines)
ranking_B <- ranking_table(fit_full, engines)
```

Table 1: Model A: Bayesian Engine Ranking (Binary Outcomes)

| Engine | Mean Rating | SD | Lower 90% | Upper 90% |
|---|---|---|---|---|
| E | 2680.9 | 123.8 | 2478.0 | 2888.0 |
| A | 2128.2 | 130.1 | 1909.6 | 2340.2 |
| SM | 2053.4 | 103.4 | 1883.6 | 2221.6 |
| C | 1815.2 | 157.1 | 1552.0 | 2074.1 |
| B | 1807.0 | 162.2 | 1535.5 | 2063.0 |
| D | 1497.3 | 130.0 | 1284.7 | 1706.9 |

```
# Render table
knitr::kable(
  ranking_B,
  digits = 1,
  col.names = c("Engine", "Mean Rating", "SD", "Lower 90%", "Upper 90%"),
  caption = "Model B: Bayesian Engine Ranking (Including Draws)",
  booktabs = TRUE,
  row.names = FALSE
)
```

Table 2: Model B: Bayesian Engine Ranking (Including Draws)

| Engine | Mean Rating | SD | Lower 90% | Upper 90% |
|---|---|---|---|---|
| E | 2290.5 | 84.9 | 2153.4 | 2433.7 |
| A | 2034.0 | 85.1 | 1896.2 | 2176.6 |
| SM | 2022.8 | 83.4 | 1887.5 | 2164.4 |
| C | 1932.1 | 89.5 | 1787.4 | 2085.1 |
| B | 1903.0 | 91.4 | 1753.7 | 2057.2 |
| D | 1784.7 | 85.5 | 1645.6 | 1928.6 |

In both the complex and the simpler model we get the same ranking order. Notice as

$$R_j - Ri = (R_j + m) - (R_i + m)$$

the absolute value of a rating isn't that informative.

## Pairwise Superiority

```r
# Calculate pairwise superiority probabilities from posterior samples
superiority_table <- function(fit, engines) {

  # Extract posterior rating samples for all engines
  draws <- as.matrix(fit$draws(
    variables = paste0("rating[", seq_along(engines), "]"),
    format = "draws_matrix"
  ))

  # Number of engines
  K <- length(engines)

  # Initialize matrix for pairwise superiority probabilities
  probs <- matrix(
    NA_real_, K, K,
    dimnames = list(engines, engines)
  )

  # Compare each engine with every other engine
  for (i in seq_len(K)) {
    for (j in seq_len(K)) {
      if (i != j) {

        # Proportion of posterior samples where engine i
        # has a higher rating than engine j
        probs[i, j] <- mean(draws[, i] > draws[, j])
      }
    }
  }

  return(probs)
}

# Calculate superiority probabilities for both models
superiority_A <- superiority_table(fit_binary, engines)
superiority_B <- superiority_table(fit_full, engines)
```

Table 3: Model A: Pairwise superiority

| | A | B | C | D | E | SM |
|---|---|---|---|---|---|---|
| A | NA | 0.960 | 0.950 | 1.000 | 0 | 0.747 |
| B | 0.040 | NA | 0.486 | 0.944 | 0 | 0.066 |
| C | 0.050 | 0.513 | NA | 0.956 | 0 | 0.060 |
| D | 0.000 | 0.056 | 0.044 | NA | 0 | 0.000 |

| | A | B | C | D | E | SM |
|---|---|---|---|---|---|---|
| E | 1.000 | 1.000 | 1.000 | 1.000 | NA | 1.000 |
| SM | 0.253 | 0.934 | 0.940 | 1.000 | 0 | NA |

Table 4: Model B: Pairwise superiority

| | A | B | C | D | E | SM |
|---|---|---|---|---|---|---|
| A | NA | 0.993 | 0.988 | 1.000 | 0 | 0.666 |
| B | 0.007 | NA | 0.312 | 0.988 | 0 | 0.004 |
| C | 0.012 | 0.688 | NA | 0.999 | 0 | 0.011 |
| D | 0.000 | 0.013 | 0.001 | NA | 0 | 0.000 |
| E | 1.000 | 1.000 | 1.000 | 1.000 | NA | 1.000 |
| SM | 0.334 | 0.996 | 0.989 | 1.000 | 0 | NA |

## Tier List

We define superiority as a posterior probability of at least 95% that one engine has a higher rating than another.

| Tier | Engines |
|---|---|
| 1 | E |
| 2 | A, SM |
| 3 | C, B |
| 4 | D |

Both models yield the same ranking: E > A > SM > C > B > D.

Model B supports the proposed tier boundaries, while differences within Tiers 2 and 3 remain uncertain. Model A provides weaker evidence for some tier boundaries. Therefore, the tier classification is based on Model B.